

TALON

On-chain transfer agent for Cleanverse-verified real-world assets

Cleanverse Build: Trusted Assets Hackathon · RWA track · solo build · contracts live & frozen on Monad testnet

Live app [talon-cleanverse-hackathon.vercel.app](#) Demo video [youtu.be/CT_b-7mUf2U](#) Code [github.com/hoBabu1/Talon-Cleanverse-Hackathon](#)

PROBLEM

eligibility drift — the gap between record date and pay date

Every RWA project stops at *issuance*. Nobody runs what happens *after*: coupons, dividends, redemptions. And post-issuance hides a failure mode unique to verified assets.

A bondholder is verified on the **record date**. The issuer declares a coupon. By the **pay date** — days later — that holder's A-Pass has expired, or their bank has frozen it. The issuer now has two options and **both are wrong**:

- **Pay them anyway** → a compliance violation. Cleanverse's own A-Token won't even permit it; the transfer reverts on-chain.
- **Withhold the coupon** → theft. They were fully verified when they *earned* it. An expired document is not a forfeiture of property.

This problem exists only because CVI credentials are revocable and expiring. On a plain ERC-20 there is nothing to drift — which is why Talon could not have been built on anything but Cleanverse.

SOLUTION

pay the compliant, preserve the lapsed, release on re-verification

live cap table → **declare** corporate action at a record block → **re-verify every holder at pay date** → **pay directly** where compliant → **escrow per-beneficiary** where not, tagged with the on-chain reason → **release** the moment they re-verify → **full audit export** with Travel Rule reports

The core design decision: Talon never re-implements compliance. It keeps no allowlist of its own. At pay date it simply attempts the *real* A-Token transfer and lets Cleanverse's contract revert by itself. Escrow release is a **retry of that same real transfer** — so if it succeeds, the holder is genuinely eligible again, as judged by Cleanverse, not by us. The chain is the source of truth; we only react to it.

Honest positioning. Corporate-actions machinery does exist — Polymesh's pallet, Securitize's DS Protocol. What doesn't exist: preserving a lapsed holder's entitlement *per beneficiary*, separating *expired* from *frozen* as distinct on-chain reasons, and attributing the eventual release with a Travel Rule report. Polymesh conflates lapsed with sanctioned and forfeits unclaimed value to the issuer at expiry. **Talon never forfeits — there is deliberately no admin-confiscation function in the contract.**

CVI · CVA INTEGRATION POINTS

six, all live against the real sandbox — none mocked

A-Pass (CVI)	The revocable, expiring credential that the entire product is a reaction to. Every holder in the register carries one.
A-Token (CVA)	The asset and the coupon currency. Its on-chain revert is our compliance check — we never second-guess it.
generate_apass	The escrow vault can custody A-Tokens only because it holds its own A-Pass — a contract is a party too.
query_apass / verify_apass	The eligibility poller. Cleanverse ships no webhook for status changes, so we poll and reconcile.
update_status	The issuer freezes or reinstates a holder from the dashboard , and we wait for the chain to enforce it before reporting success.
download_travel_rule	Per-payout regulatory report, participation-verified before request, bytes stored — not just a URL that may expire.

Depth, not surface — five things only real integration teaches:

- **Three distinct failure modes, never conflated.** APassNotActive 0x322fde89 (frozen), APassExpired 0xaecc0dbe (expired), NoAPass 0xa6725971. Expired carries its own on-chain reason tag — folding it into "frozen" would make our own data contradict our claim that *lapsed* ≠ *sanctioned*.
- **Revert data names the offending party.** So a holder failure escrows *that holder*, while an issuer- or vault-side failure reverts the whole batch — one party's problem is never misattributed to another's.
- **Senders are checked too**, so the vault's own A-Pass is a live dependency — the poller watches it, and our own admin route refuses to freeze the vault or the issuer, since either would halt every payout in the register.
- **download_travel_rule does not verify participation** — it returns a valid-looking report for a wallet that never appeared in the transaction. We check the transaction's own Transfer logs first.
- **A 200 from a Cleanverse write is not proof of on-chain state.** generate_apass returns success while verify_apass still reports inactive for seconds afterwards. Every write goes through one helper that waits for the chain to agree before anything is recorded.

DEPLOYED CHAIN

Monad testnet · chain ID 10143 · contracts deployed and frozen

EscrowVault — per-beneficiary escrow engine	0xb634379B2afdF12830eaeF694cFeaE80fB0dFFB7
CorporateActionManager — declares & pays	0xF897874bAe28443a60ef92741f7df504F90386b6
TLNB "Talon Bond 2026" — the asset (A-Token)	0xbAE642890988C3EF56e77Fb041aFD847A6131d64
aUSDC — the coupon currency (A-Token)	0xaC0893567D43C3E7e6e35a72803df05416C1f20D

PROOF & SCALABILITY

verified against the real chain, real sandbox, real database

- **90 contract tests green** — unit, fuzz, and cross-contract invariants (256 runs × depth 64) proving `sum(ledger) ≤ balanceOf(vault)` through the real manager → vault wiring.
- **Proven with money that moves.** A live smoke test on **real aUSDC on real Monad testnet**: one holder paid directly; a second frozen → escrowed → unfrozen → released → claimed. Drift and recovery, end to end.
- **64 live assertions** across chain + Cleanverse + database, checking after every step that **the chain and the read-mirror agree field by field** — because a bug once lived in exactly that seam and returned a clean 200.
- **One vault, every asset.** A single vault custodies any number of A-Tokens (beneficiary → token → amount). No redeploy per issuance.
- **Resumable batched execution.** A payout run spans as many transactions as the register needs and survives interruption; paginated enumeration means no unbounded loop can brick a view.
- **Chained-hash commitment** seeded with a provenance binding, so a committed batch cannot be replayed against a different action, token, or deployment.
- **Event-sourced.** Off-chain state rebuilds from logs; the database is a read-mirror, never a second source of truth.